ЛАБОРАТОРНА РОБОТА №3

Тема: Часткові представлення. Робота з View Components. Навігація. Фільтрація. Використання сесії.

Мета: отримати навички роботи з частковими представленнями, навчитися працювати з параметрами рядку запиту, набути навичок роботи з сесіями.

Хід роботи:

Завдання 1. Створення часткового представлення (Partial View)

1. У папці Views/Shared створіть .cshtml файл для одиничного елементу зі списку сутностей (для SportsStore це буде ProductSummary.cshtml)

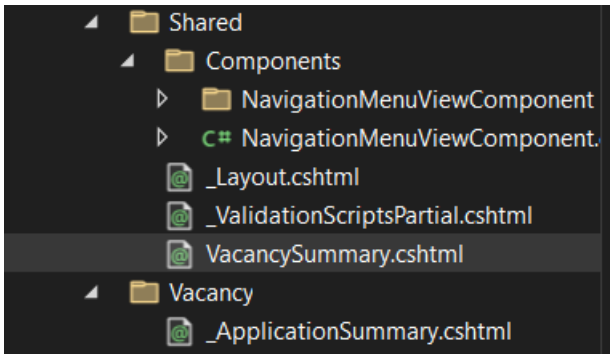

Рис. 1. Файл для одиночого елементу.

2. Перемістіть код для одиничного елементу з загального View Index.cshtm в створений файл.

Лістинг VacancySummary.cshtml:

```
@model Lab3.Models.ViewModels.VacancyViewModel

<div class="product-item">
    <div>
        <h3>@Model.Title</h3>
        <p>Description: @Model.Description</p>
        <p>Company: @Model.Company</p>
        <p>Salary: @Model.Salary.ToString("N2") €</p>
        <p>Category: @Model.Category</p>
    </div>
    <a class="btn btn-success" asp-controller="Vacancy" asp-action="Apply" asp-route-
id="@Model.VacancyId">Apply</a>
   </div>
```

3. Використайте створений Partial View в Index.cshtml за допомогою Tag Helper’a <partial>

Лістинг Index.cshtm:

```
@model Lab3.Models.ViewModels.VacanciesListViewModel

<div class="row">
    <div class="col-12">
        <h2>Vacancies</h2>
        @foreach (var vacancy in Model.Vacancies)
        {
            <partial name="VacancySummary" model="vacancy" />
        }
    </div>
</div>

<div class="row">
    <div class="col-12 text-center">
        @if (Model.PagingInfo.TotalPages > 1)
        {
            <nav aria-label="Page navigation">
                <ul class="pagination d-inline-flex">
                    @for (int i = 1; i <= Model.PagingInfo.TotalPages; i++)
                    {
              <li class="page-item @(i == Model.PagingInfo.CurrentPage ? "active" : "")">
<a class="page-link" asp-action="Index" asp-route-page="@i" asp-route-category="@Model.
CurrentCategory">@i</a>
                        </li>
                    }
                </ul>
            </nav>
        }
    </div>
   </div>
```

Завдання 2. Навігація по категоріям

Реалізуйте навігацію за категорією (або за іншою ознакою, яка більше підходить за вашим варіантом), дотримуючись наступних кроків:

1. В папці проєкту створіть папку Components, в ній створіть файл з назвою NavigationMenuViewComponent.cs та реалізуйте його.

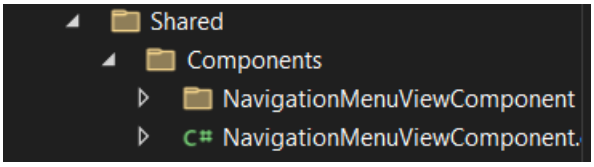

Рис. 2. Папка з відповідним файлом.

Лістинг NavigationMenuViewComponent.cs:

```
using Microsoft.AspNetCore.Mvc;
using Lab3.Models;

namespace Lab3.Components
{
    [ViewComponent(Name = "NavigationMenuViewComponent")]
    public class NavigationMenuComponent : ViewComponent
    {
        private readonly IJobRepository repository;
```

```
        public NavigationMenuComponent(IJobRepository repo)
        {
            repository = repo;
        }

        public async Task<IViewComponentResult> InvokeAsync()
        {
            var categories = repository.Vacancies
                .Select(v => v.Category)
                .Distinct()
                .ToList();

            string candidateId = "candidate-id";
            var applicationCount = repository.Applications
                .Count(a => a.CandidateId == candidateId);

            var viewModel = new NavigationMenuViewModel
            {
                Categories = categories,
                ApplicationCount = applicationCount
            };

Console.WriteLine($"NavigationMenuViewComponent: Found {categories.Count} categories:
{string.Join(", ", categories)}");
            return View(viewModel);
        }
    }

    public class NavigationMenuViewModel
    {
        public IEnumerable<string> Categories { get; set; }
        public int ApplicationCount { get; set; }
    }
}
```

2. В загальному шаблоні (файл _Layout.cshtml) додайте розмітку для сайд-бару. В доданій розмітці додайте виклик створеного NavigationMenuViewComponent.

Лістинг (частковий) _Layout.cshtml:

```
<div class="container-fluid">
    <div class="row">
        <div class="col-md-2 category-nav">
            @await Component.InvokeAsync("NavigationMenuViewComponent")
        </div>

        <div class="col-md-10">
            <div class="container mt-3">
                @RenderBody()
            </div>
        </div>
    </div>
      </div>
```

3. Змініть реалізацію NavigationMenuViewComponent таким чином, щоб компонент отримував перелік категорій з БД і повертав результат типу IViewComponentResult.

Лістинг (частковий) NavigationMenuViewComponent.cs:

```
public async Task<IViewComponentResult> InvokeAsync()
{
    var categories = repository.Vacancies
        .Select(v => v.Category)
        .Distinct()
        .ToList();

    string candidateId = "candidate-id";
    var applicationCount = repository.Applications
        .Count(a => a.CandidateId == candidateId);

    var viewModel = new NavigationMenuViewModel
    {
        Categories = categories,
        ApplicationCount = applicationCount
    };

    Console.WriteLine($"NavigationMenuViewComponent: Found {categories.Count} categories:
{string.Join(", ", categories)}");
    return View(viewModel);
}
```

4. В папці Views/Shared/Components/NavigationMenu створіть файл Default.cshtml.

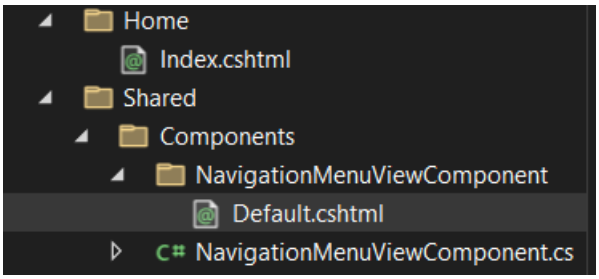

Рис. 3. Папка з відповідним файлом.

5. В створеному файлі представлення реалізуйте виведення списку посилань, отриманих з ViewComponent'а.

Лістинг Default.cshtml:

```
@model Lab3.Components.NavigationMenuViewModel

<div class="d-flex flex-column">
    <a class="nav-link" asp-controller="Vacancy" asp-action="MyApplications">

        My Applications

        @if (Model.ApplicationCount > 0)
        {
            <span class="badge bg-primary">@Model.ApplicationCount</span>
        }
    </a>

    <a class="nav-link
@(string.IsNullOrEmpty(ViewContext.RouteData.Values["category"]?.ToString()) ? "active" : "")"
   asp-controller="Vacancy" asp-action="Index" asp-route-category="">All Categories</a>
```

```
    @foreach (var category in Model.Categories)
    {
        <a class="nav-link @(ViewContext.RouteData.Values["category"]?.ToString() ==
category ? "active" : "")"
           asp-controller="Vacancy" asp-action="Index" asp-route-
category="@category">@category</a>
    }
</div>
```

6. У файлі Controllers/HomeController.cs в методі Index() реалізуйте підтримку фільтрації по категоріям.

Лістинг HomeController.cs:

```
public IActionResult Index(string category, int page = 1)
 {
     var vacancies = repository.Vacancies
         .Where(v => category == null || v.Category == category)
         .OrderBy(v => v.VacancyId);

     var totalItems = vacancies.Count();
     var pagedVacancies = vacancies
         .Skip((page - 1) * pageSize)
         .Take(pageSize)
         .Select(v => new VacancyViewModel
         {
             VacancyId = v.VacancyId,
             Title = v.Title,
             Description = v.Description,
             Company = v.Company,
             Salary = v.Salary,
             Category = v.Category
         });

     var viewModel = new VacanciesListViewModel
     {
         Vacancies = pagedVacancies,
         PagingInfo = new PagingInfo
         {
             CurrentPage = page,
             ItemsPerPage = pageSize,
             TotalItems = totalItems
         },
         CurrentCategory = category
     };

     return View("~/Views/Home/Index.cshtml", viewModel);
 }
```

7. Змініть ViewModel таким чином, щоб мати інформацію про обрану категорію на представленні. Це знадобиться для формування посилань на сторінки з урахуванням обраної категорії.

Лістинг VacancyListViewModel.cs:

```
namespace Lab3.Models.ViewModels
{
    public class VacancyViewModel
    {
        public long VacancyId { get; set; }
        public string Title { get; set; } = string.Empty;
        public string Description { get; set; } = string.Empty;
```

```
        public string Company { get; set; } = string.Empty;
        public decimal Salary { get; set; }
        public string Category { get; set; } = string.Empty;
    }

    public class VacanciesListViewModel
    {
        public IEnumerable<VacancyViewModel> Vacancies { get; set; }
        public PagingInfo PagingInfo { get; set; }
        public string CurrentCategory { get; set; }
    }
}
```

8. Реалізуйте виділення категорії, обраної користувачем. Для цього передавайте інформацію про обрану категорію з View Component в View.

Лістинг (частковий) NavigationMenuViewComponent.cs:

```
public async Task<IViewComponentResult> InvokeAsync()
{
    var categories = repository.Vacancies
        .Select(v => v.Category)
        .Distinct()
        .ToList();
    ViewBag.SelectedCategory = RouteData?.Values["category"]; //передача категорії
    return View(viewModel);
}
```

Лістинг (частковий) Default.cshtml:

```
<a class="nav-link
@(string.IsNullOrEmpty(ViewContext.RouteData.Values["category"]?.ToString()) ? "active" :
"")"
    asp-controller="Vacancy" asp-action="Index" asp-route-category="">All Categories</a>
 @foreach (var category in Model.Categories)
 {
     <a class="nav-link @(ViewContext.RouteData.Values["category"]?.ToString() ==
category ? "active" : "")"
        asp-controller="Vacancy" asp-action="Index" asp-route-
category="@category">@category</a>
 }
```

9. Змініть реалізацію пейджингу таким чином, щоб він відображав кількість доступних сторінок відповідно до обраної категорії.

Лістинг (частковий) Index.cshtm:

```
@for (int i = 1; i <= Model.PagingInfo.TotalPages; i++)
 {
     <li class="page-item @(i == Model.PagingInfo.CurrentPage ? "active" : "")">
         <a class="page-link" asp-action="Index" asp-route-page="@i"
asp-route-category="@Model.CurrentCategory">@i</a>
     </li>

        }
```

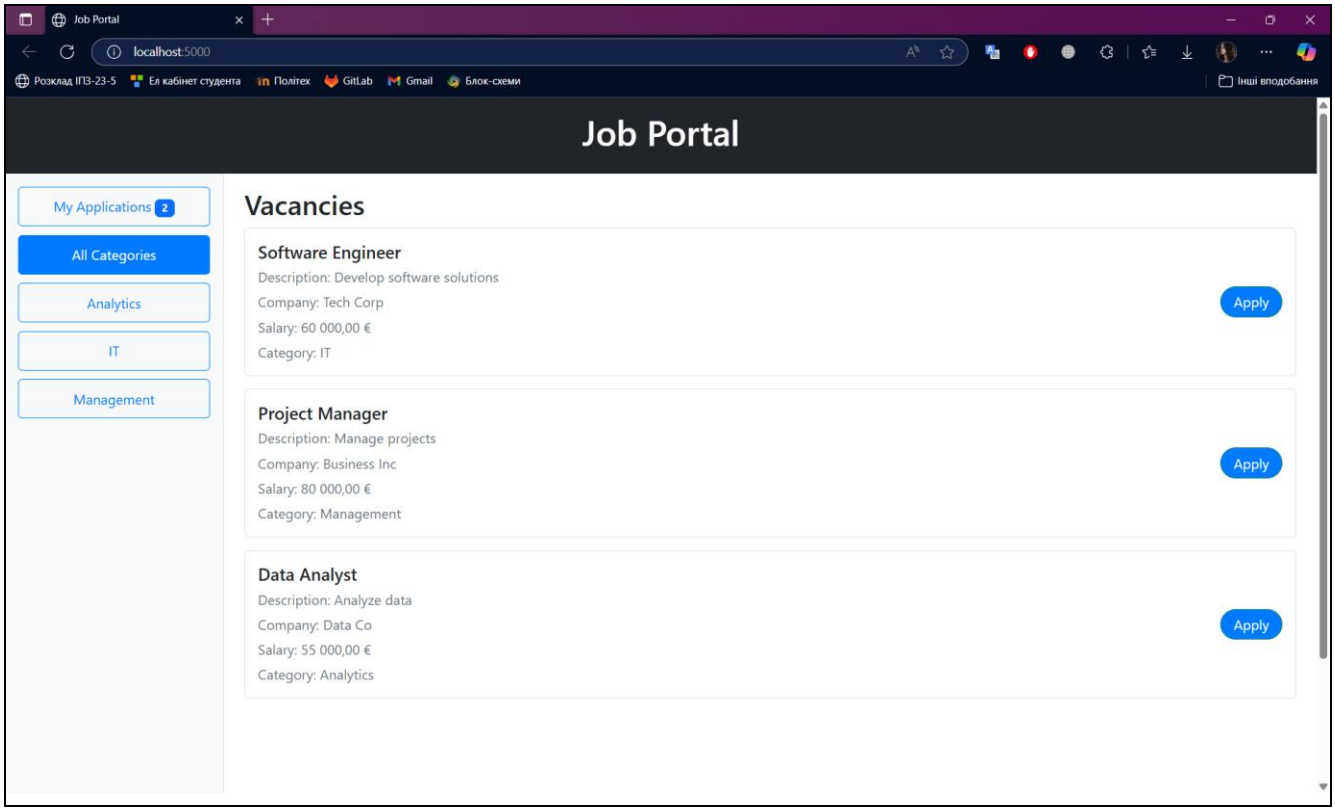

Рис. 4. Вигляд сторінки.

Рис. 5. Сортування по категорії: IT.

Завдання 3. Сесії

Реалізуйте функціональність відповідно до варіанту вашого проекту з використанням механізму сесії (табл. 3.1.).

Номер за списком	Тема проєкту	Опис функціоналу
3	Портал вакансій	Збереження незавершених заявок на вакансії у сесії, щоб кандидат міг продовжити заповнення форми після переривання.

Лістинг (частковий) EditApplication.cshtml:

```
@section Scripts {
    <script>
        const resumeInput = document.getElementById("resumeInput");
        const draftStatus = document.getElementById("draftStatus");
        let debounceTimer;

        resumeInput.addEventListener("input", function () {
            clearTimeout(debounceTimer);
            draftStatus.textContent = "Saving draft...";

            debounceTimer = setTimeout(() => {
                const formData = new FormData(document.getElementById("editForm"));
                fetch('/Vacancy/SaveEditDraft', {
                    method: 'POST',
                    body: formData
                })
                .then(response => response.json())
                .then(data => {
                    if (data.success) {
  draftStatus.textContent = "Draft saved at " + new Date().toLocaleTimeString();
                    } else {
                        draftStatus.textContent = "Error saving draft";
                    }
                })
                .catch(error => {
                    console.error("Error:", error);
                    draftStatus.textContent = "Error saving draft";
                });
            }, 1000);
        });
    </script>
      }
```

Лістинг (частковий) Apply.cshtml:

```
@section Scripts {
    <script>
        const resumeInput = document.getElementById("resumeInput");
        const draftStatus = document.getElementById("draftStatus");
        let debounceTimer;
        resumeInput.addEventListener("input", function () {
            clearTimeout(debounceTimer);
            draftStatus.textContent = "Saving draft...";
            debounceTimer = setTimeout(() => {
                const formData = new FormData(document.getElementById("applyForm"));
                fetch('/Vacancy/SaveDraft', {
                    method: 'POST',
                    body: formData})
                .then(response => response.json())
                .then(data => {
                    if (data.success) {
draftStatus.textContent = "Draft saved at " + new Date().toLocaleTimeString();
                    } else {
                        draftStatus.textContent = "Error saving draft";
                    }})
                .catch(error => {
                    console.error("Error:", error);
                    draftStatus.textContent = "Error saving draft";
                });
            }, 1000);
        });
    </script>
      }
```

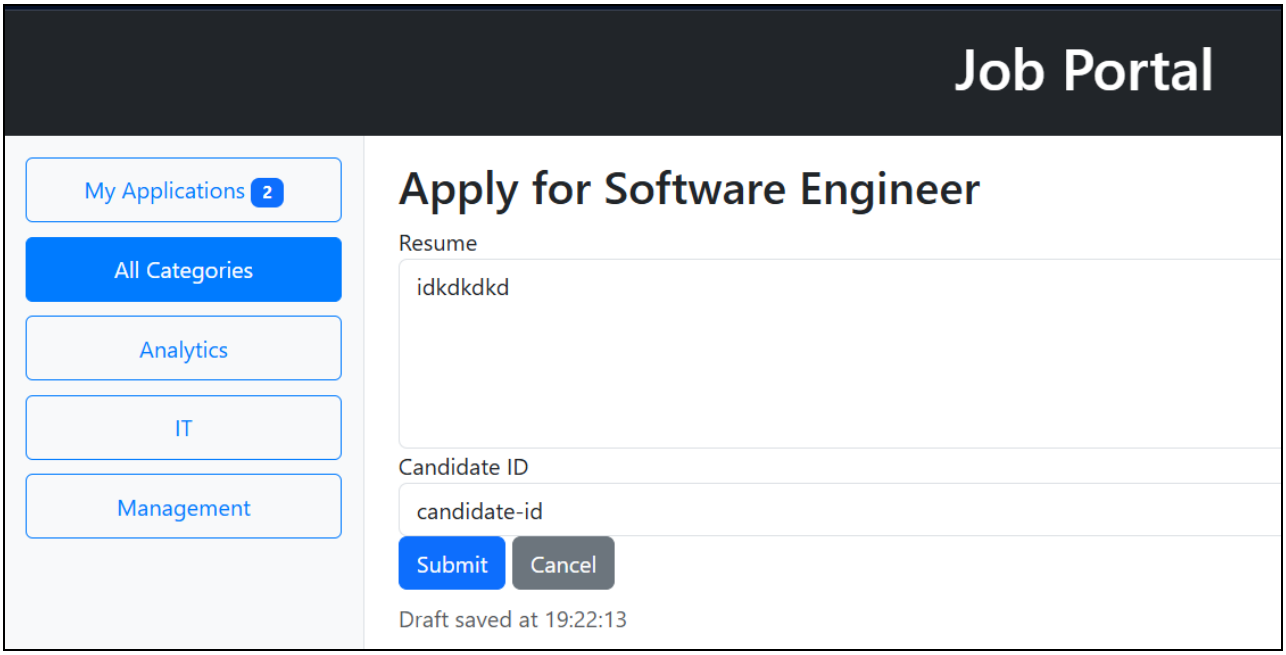

Рис. 6. Збереження у буфері при подачі заявки.

При перезавантаженні сторінки або перехід на іншу та повернення на цю вакансію текст залишається.

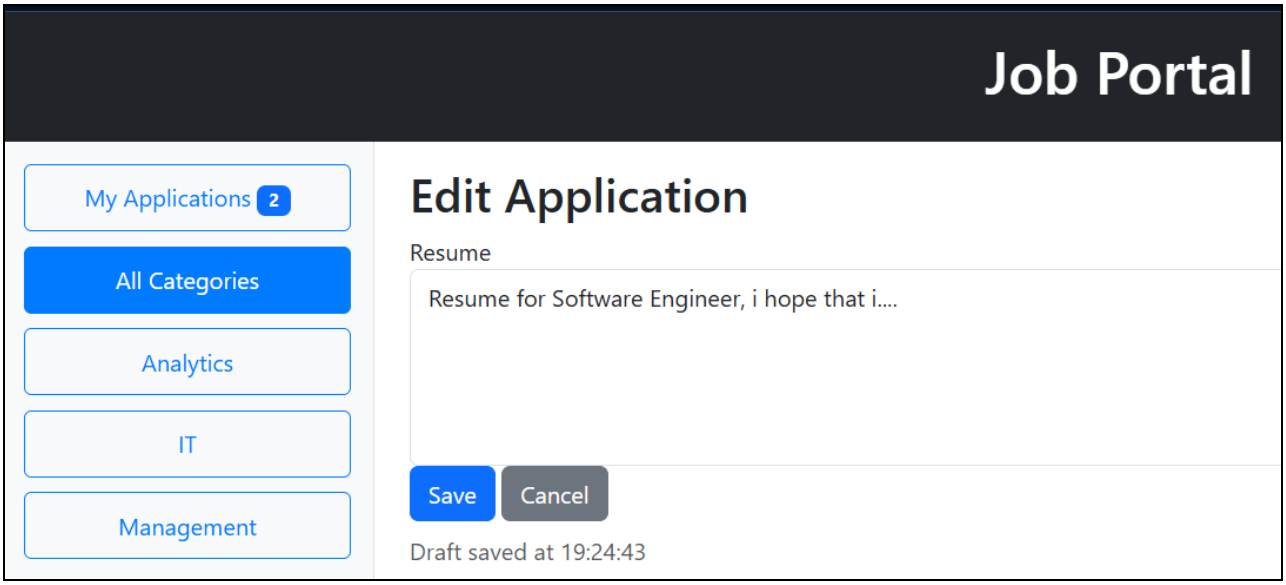

Рис. 7. Збереження у буфері при редагуванні заявки.

Тут як і в попередньому. Проте на сторінці заявок текст у буфері не відображається, лише з'являється в режимі редагування.

Висновок: в ході лабораторної роботи №3 я отримала навички роботи з частковими представленнями та навчилася працювати з параметрами рядку запиту, набула навичок роботи з сесіями.

Посилання на репозиторій:

IPZ / 2023-2027 / IPZ-23-5 / Bubon Lina / ASP.NET · GitLab